



RÉPUBLIQUE TUNISIENNE

MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE
SCIENTIFIQUE

École Supérieure Privée d'Ingénierie et de Technologies —
ESPRIT

RAPPORT DE STAGE D'IMMERSION EN ENTREPRISE

ATB Credit Platform

Plateforme de gestion de dossiers de crédit
avec scoring de risque par intelligence artificielle explicable

Élaboré par :
Mohamed Azer Khadhraoui

Encadrante :
Wejden Mhamdi

Organisme d'accueil : Arab Tunisian Bank (ATB)

Année universitaire 2025-2026

Table des matières

Introduction générale	1
1 Cadre général du projet	3
1.1 Organisme d'accueil	3
1.2 Cadre du stage	3
1.3 Étude de l'existant	3
1.4 Solution proposée	4
1.5 Méthodologie de travail	4
2 Analyse et spécification des besoins	6
2.1 Identification des acteurs	6
2.2 Besoins fonctionnels	7
2.3 Besoins non fonctionnels	8
2.4 Cas d'utilisation	8
3 Conception	11
3.1 Architecture générale	11
3.2 Modèle de données	11
3.3 Conception des interfaces	13
4 Réalisation	15
4.1 Environnement de travail	15
4.2 Technologies retenues	16
4.3 Sécurité applicative	16
4.4 Stratégie de tests	17
4.5 Interfaces réalisées	18
5 Module d'apprentissage automatique	21
5.1 Jeu de données	21
5.2 Préparation des données	22
5.3 Choix du modèle	22
5.4 Explicabilité des décisions	22
5.5 Intégration au backend	24
6 Démarche DevSecOps et déploiement	26
6.1 Conteneurisation	26

6.2	Intégration continue	27
6.3	Sécurité intégrée à la chaîne	27
6.4	Déploiement sur Kubernetes	28
6.5	Infrastructure décrite en code	29
6.6	Supervision applicative	30
6.7	Publication des images	31
Conclusion générale		32
Bibliographie		34

Table des figures

2.1	Diagramme de cas d'utilisation général	10
3.1	Architecture microservices de la plateforme	11
3.2	Diagramme de classes de la plateforme	12
4.1	Écran de connexion	18
4.2	Écran de création de compte, avec validation par champ	19
4.3	Détail d'un dossier et panneau d'analyse avec explicabilité	19
4.4	Journal d'audit — traçabilité des connexions, créations et analyses . . .	20
6.1	Exécution de la chaîne d'intégration continue et des analyses de sécurité	28
6.2	Tableau de bord de supervision du backend	30

Liste des tableaux

1.1	Limites du traitement non outillé et réponses apportées	4
2.1	Acteurs du système	6
2.2	Besoins fonctionnels par module	7
2.3	Besoins non fonctionnels	8
2.4	Description du cas d'utilisation « Analyser un dossier »	9
3.1	Collections et attributs principaux	13
3.2	Principaux écrans	14
4.1	Environnement logiciel	15
4.2	Choix technologiques et justifications	16
4.3	Couverture de tests par service	17
5.1	Variables du jeu de données	21
5.2	Explication produite pour un dossier à risque élevé (score 73,68)	24
5.3	Explication produite pour un dossier à risque faible (score 9,60)	24
6.1	Étapes de la chaîne d'intégration continue	27
6.2	Outils d'analyse et périmètres couverts	27
6.3	Objets Kubernetes déployés et justifications	29

Introduction générale

Contexte

L'octroi de crédit constitue une activité centrale de l'activité bancaire, et l'évaluation du risque associé à chaque demande en conditionne directement la rentabilité. Cette évaluation repose traditionnellement sur l'expertise de l'agent de crédit, qui analyse manuellement les éléments du dossier : revenus, charges, historique de remboursement, garanties apportées.

Ce mode de traitement présente trois limites structurelles. D'abord, la **durée** : chaque dossier mobilise un temps d'analyse important. Ensuite, la **variabilité** : deux agents confrontés au même dossier peuvent aboutir à des conclusions différentes, faute de critère commun quantifié. Enfin, la **traçabilité** : lorsque la décision et sa justification ne sont pas consignées de manière structurée, il devient difficile de les auditer a posteriori.

L'apprentissage automatique offre une réponse partielle à ces limites, en produisant un score reproductible à partir de critères objectifs. Mais il introduit un problème nouveau : celui de l'**opacité**. Un modèle qui énonce un verdict sans le motiver est inutilisable dans un contexte bancaire, où l'agent doit pouvoir expliquer sa décision au client et à sa hiérarchie.

Problématique

Ce constat conduit à la question directrice de ce projet :

Comment assister l'agent de crédit dans son évaluation du risque, par un modèle statistique reproductible, sans le déposséder de la décision finale ni lui imposer un verdict qu'il ne peut ni comprendre ni justifier ?

La réponse retenue ne consiste pas à automatiser la décision, mais à l'**outiller** : le modèle propose un score et une recommandation, et surtout expose les critères qui les ont déterminés. L'agent conserve la décision ; il gagne un argumentaire.

Objectifs

Le projet vise à concevoir et réaliser une plateforme web permettant :

- la gestion centralisée des clients et de leurs dossiers de crédit ;
- le calcul d'un score de risque assisté par apprentissage automatique ;
- l'explicitation de chaque décision, critère par critère ;
- la traçabilité complète des actions sensibles ;
- un déploiement automatisé et sécurisé, conforme aux pratiques DevSecOps.

Organisation du rapport

Ce rapport s'articule en six chapitres : le cadre général du projet, l'analyse et la spécification des besoins, la conception de la solution, sa réalisation, le module d'apprentissage automatique et son explicabilité, et enfin la démarche DevSecOps accompagnée du déploiement.

Chapitre 1

Cadre général du projet

1.1 Organisme d'accueil

L'Arab Tunisian Bank (ATB) est une banque commerciale tunisienne, filiale du groupe Arab Bank. Elle propose une gamme complète de services bancaires destinés aux particuliers, aux professionnels et aux entreprises, parmi lesquels les activités de crédit occupent une place centrale.

C'est au sein de cette activité que s'inscrit le présent projet : l'évaluation du risque attaché à une demande de crédit conditionne à la fois la qualité du portefeuille de la banque et le délai de réponse apporté au client.

1.2 Cadre du stage

Ce projet a été réalisé dans le cadre d'un **stage d'immersion en entreprise** au sein de l'Arab Tunisian Bank, durant les mois de **juillet et août 2026**, sous l'encadrement de Madame Wejden Mhamdi.

Ce type de stage vise la découverte du milieu professionnel et de ses méthodes de travail. Il a ici pris la forme d'un projet applicatif complet, conduit de l'analyse du besoin jusqu'au déploiement, ce qui a permis de confronter les acquis théoriques aux contraintes réelles d'un système d'information bancaire : exigences de sécurité, traçabilité des opérations et justification des décisions.

1.3 Étude de l'existant

Le traitement d'un dossier de crédit, dans un processus non outillé, suit généralement les étapes suivantes : collecte des pièces auprès du client, saisie des informations, analyse de la capacité de remboursement, puis formulation d'un avis transmis au comité de décision.

Cette organisation présente plusieurs limites, qui constituent les axes d'amélioration retenus pour le projet :

Table 1.1 – Limites du traitement non outillé et réponses apportées

Limite constatée	Réponse du projet
Données client et dossiers dispersés entre plusieurs supports, sans référentiel unique.	Base de données centralisée, un client pouvant porter plusieurs dossiers reliés.
Évaluation du risque reposant sur l’appréciation individuelle, sans critère quantifié partagé.	Score calculé par un modèle unique, appliquant les mêmes coefficients à tous les dossiers.
Absence d’historique consultable des actions et de leurs auteurs.	Journal d’audit horodaté, enregistrant chaque opération sensible avec son auteur.
Difficulté à produire une vue d’ensemble de l’activité.	Tableau de bord agrégeant volumétrie, répartition par statut et par niveau de risque.

1.4 Solution proposée

La solution retenue est une application web interne, structurée en architecture microservices, destinée aux agents de crédit et aux administrateurs. Elle centralise la gestion des clients et des dossiers, et intègre un moteur d’analyse de risque fondé sur l’apprentissage automatique.

Le principe directeur est celui de l’**assistance à la décision** : le modèle produit un score et une recommandation, mais la décision finale demeure celle de l’agent. Cette orientation justifie l’effort particulier porté sur l’explicabilité, développée au chapitre 5.

1.5 Méthodologie de travail

Le développement a suivi une démarche itérative, module par module, chaque itération produisant une fonctionnalité utilisable avant de passer à la suivante. L’ordre retenu — authentification, gestion des utilisateurs, des clients, des dossiers, puis analyse de risque — reflète les dépendances fonctionnelles : un dossier n’a de sens que rattaché à un client, lequel n’est saisissable que par un utilisateur authentifié.

La gestion de version s’appuie sur Git, selon le schéma suivant :

- une branche par fonctionnalité, isolant les travaux en cours ;
- intégration dans la branche principale par *pull request* ;

- exécution automatique des tests et des analyses de sécurité avant fusion ;
- suppression de la branche une fois intégrée.

Ce fonctionnement garantit que la branche principale reste à tout moment dans un état vérifié.

Chapitre 2

Analyse et spécification des besoins

2.1 Identification des acteurs

Table 2.1 – Acteurs du système

Acteur	Rôle
Agent de crédit	Gère les clients et les dossiers, déclenche l'analyse de risque, consulte l'explication produite et formule un avis motivé.
Administrateur	Dispose de l'ensemble des droits de l'agent, et gère en outre les comptes utilisateurs et consulte le journal d'audit.
Service d'analyse	Acteur non humain : reçoit les caractéristiques d'un dossier, retourne un score, un niveau de risque, une décision indicative et les facteurs qui les expliquent.

2.2 Besoins fonctionnels

Table 2.2 – Besoins fonctionnels par module

Module	Fonctionnalités
Authentification	Connexion par matricule et mot de passe ; session sans état ; contrôle d'accès différencié selon le rôle ; déconnexion.
Utilisateurs	Création, consultation, modification et suppression des comptes ; attribution du rôle ; dépôt d'une photo de profil ; changement de mot de passe ; page de profil personnel accessible à chaque utilisateur pour ses propres données.
Clients	Gestion complète du référentiel : état civil, coordonnées, situation familiale et professionnelle, revenus et charges, ainsi que les critères exploités par le modèle (personnes à charge, niveau d'études, zone d'habitation). Unicité garantie par le numéro de carte d'identité.
Dossiers de crédit	Création d'un dossier rattaché à un client ; saisie du type de crédit, du montant, de la durée, de l'objet, du taux, de l'apport, des crédits en cours, de la mensualité, des garanties et de l'historique de remboursement ; suivi du statut ; consultation détaillée.
Analyse de risque	Déclenchement à la demande sur un dossier ; restitution d'un score de 0 à 100, d'un niveau (<i>faible, moyen, élevé</i>), d'une décision indicative (<i>favorable, à surveiller, défavorable</i>) et des critères déterminants classés par influence.
Journal d'audit	Enregistrement automatique des connexions, créations, modifications, suppressions et analyses ; consultation réservée aux administrateurs.
Tableau de bord	Indicateurs consolidés : nombre de clients, de dossiers et d'utilisateurs, montant total engagé, score moyen, répartition par statut, par niveau de risque et par type de crédit, activité récente.

2.3 Besoins non fonctionnels

Table 2.3 – Besoins non fonctionnels

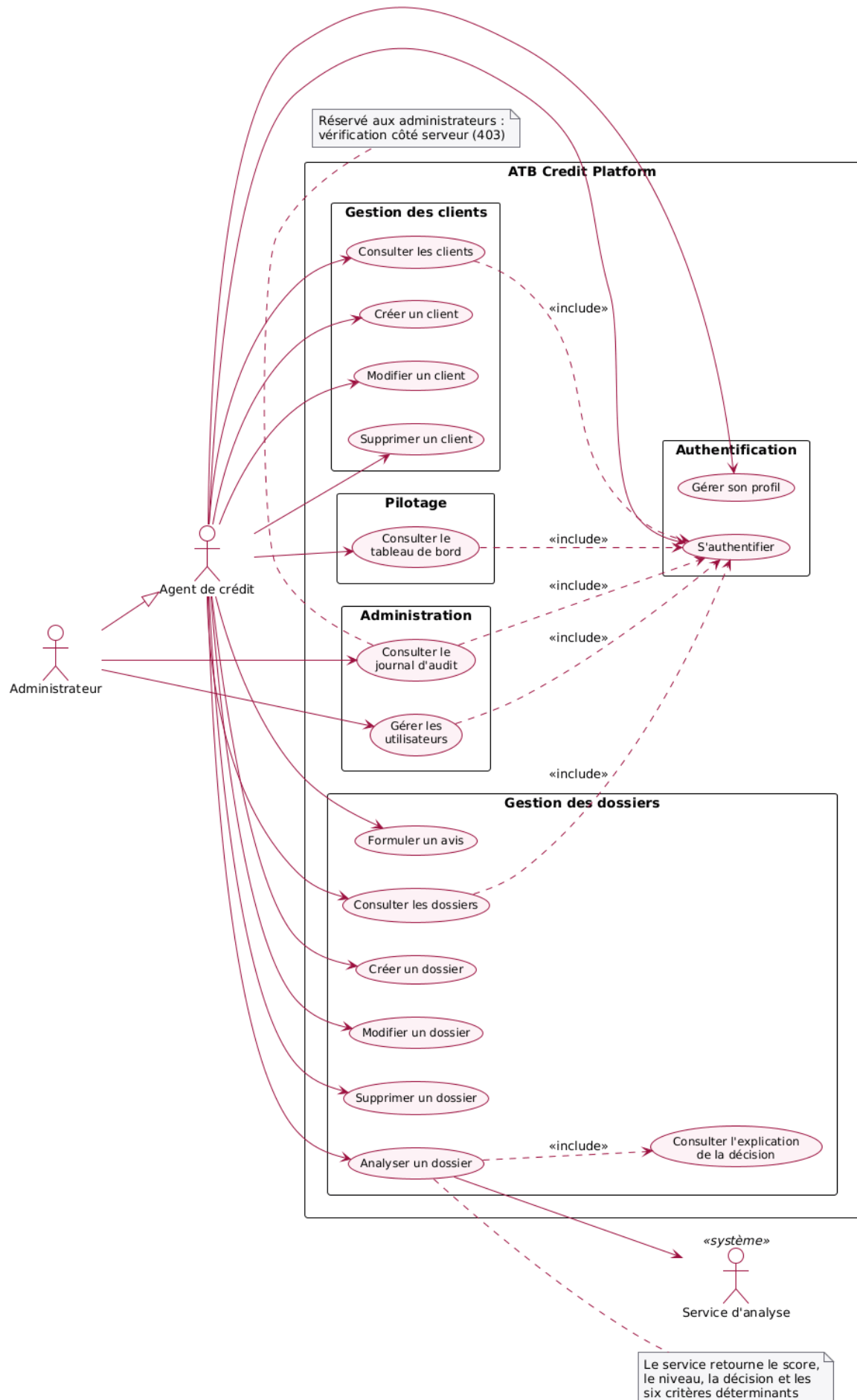
Catégorie	Exigence
Sécurité	Authentification sans état par jeton signé ; mots de passe hachés avec une fonction adaptée ; autorisations vérifiées côté serveur et non seulement masquées dans l’interface ; aucun secret présent dans le code source.
Traçabilité	Toute action sensible enregistrée avec son auteur, sa nature, l’entité concernée et son horodatage.
Explicabilité	Aucune décision du modèle ne doit être opaque : chaque score s’accompagne des critères qui l’ont déterminé et de leur sens d’influence.
Maintenabilité	Séparation stricte des responsabilités entre services ; couverture de tests sur la logique métier ; analyse statique et mesure de qualité automatisées.
Portabilité	Exécution identique en poste de développement et sur cluster, sans reconfiguration manuelle.
Ergonomie	Interface en français ; messages de validation explicites pour chaque champ ; restitution visuelle des résultats d’analyse plutôt que numérique seule.

2.4 Cas d’utilisation

Le cas d’utilisation central est l’**analyse d’un dossier de crédit**. Il se décompose ainsi :

Table 2.4 – Description du cas d'utilisation « Analyser un dossier »

Élément	Description
Acteur principal	Agent de crédit
Précondition	L'agent est authentifié ; le dossier existe et est rattaché à un client dont les données sont complètes.
Déclencheur	L'agent consulte le dossier et demande son analyse.
Scénario nominal	1. Le système rassemble les données du dossier et du client. 2. Il les convertit dans le format attendu par le modèle. 3. Il transmet la demande au service d'analyse. 4. Le service calcule le score et les facteurs explicatifs. 5. Le système enregistre le résultat sur le dossier. 6. Il consigne l'opération dans le journal d'audit. 7. L'interface affiche le score, le niveau, la décision et les critères déterminants.
Scénario alternatif	Si le service d'analyse est indisponible, le système affiche un message d'erreur explicite et le dossier reste inchangé.
Postcondition	Le dossier porte un score, un niveau, une décision et les six facteurs les plus influents ; l'opération est tracée.



Chapitre 3

Conception

3.1 Architecture générale

L'application adopte une architecture **microservices** : trois services autonomes, chacun responsable d'un domaine, communiquant par API REST.

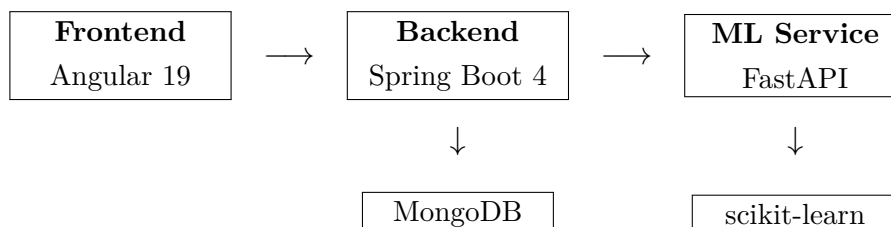


Figure 3.1 – Architecture microservices de la plateforme

Cette séparation obéit à trois principes :

- le **frontend** ne porte aucune logique métier : il consomme l'API et restitue les résultats ;
- le **backend** concentre l'authentification, les règles métier et la persistance, et orchestre l'appel au service d'analyse ;
- le **service d'analyse** ignore tout de la base de données et de l'authentification : il expose un modèle, et rien d'autre.

Ce découplage a une conséquence pratique vérifiée pendant le projet : le modèle a pu être modifié et réentraîné à plusieurs reprises sans qu'aucune ligne du backend ne change, dès lors que le contrat d'échange restait stable.

3.2 Modèle de données

La persistance repose sur MongoDB, base orientée documents. Ce choix se justifie par la nature évolutive des données de dossier : les critères exploités par le modèle ont changé au cours du projet — trois attributs ont été ajoutés au client en cours de développement — sans imposer de migration de schéma.

Diagramme de classes - Plateforme de gestion des dossiers de crédit

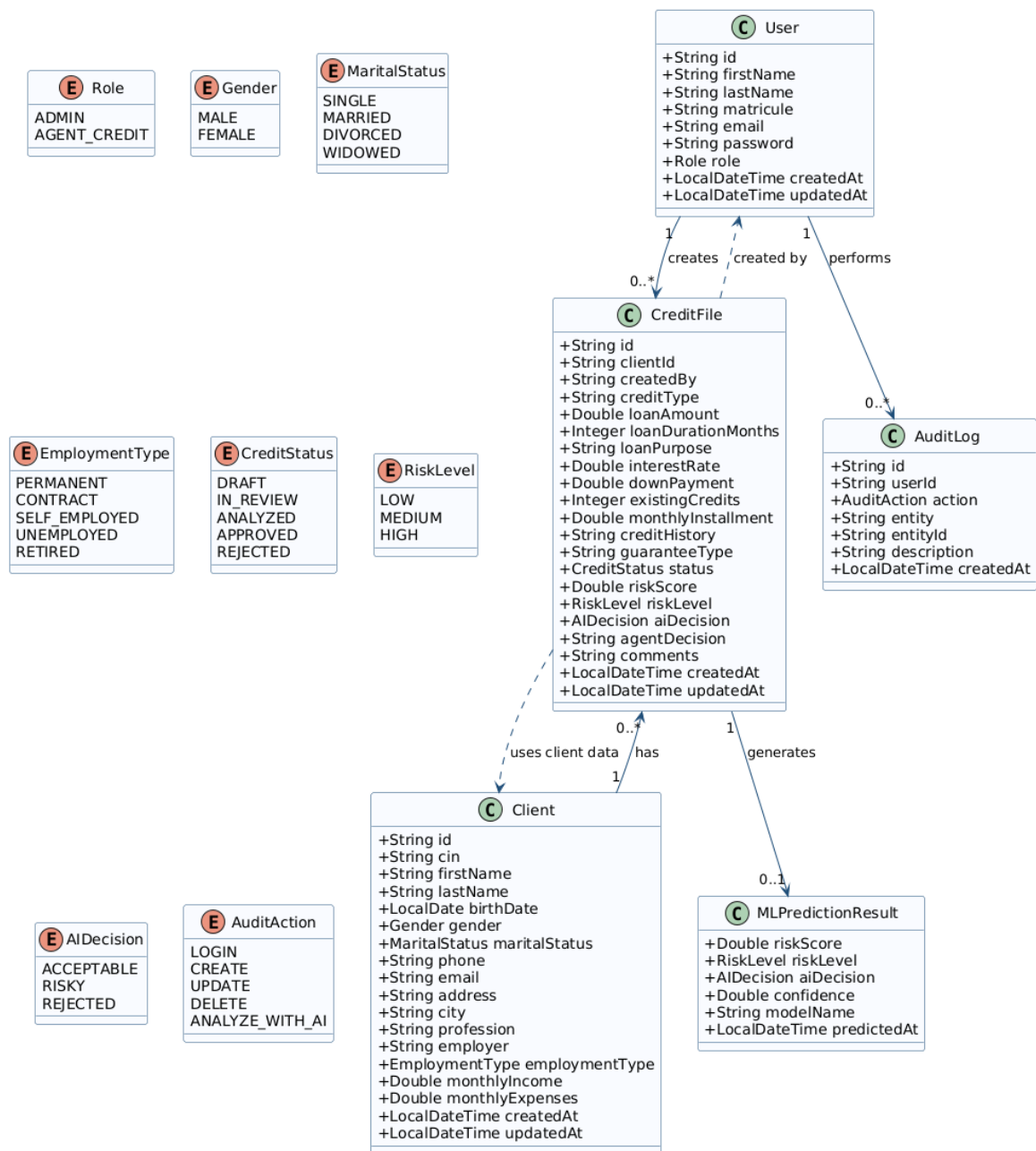


Figure 3.2 – Diagramme de classes de la plateforme

Table 3.1 – Collections et attributs principaux

Collection	Attributs
users	identifiant, matricule (unique), prénom, nom, courriel, mot de passe haché, rôle (ADMIN ou AGENT_CREDIT), photo, dates de création et de modification.
clients	identifiant, numéro de carte d'identité (unique), prénom, nom, date de naissance, genre, situation familiale, personnes à charge, niveau d'études, téléphone, courriel, adresse, ville, zone d'habitation, profession, employeur, type de contrat, revenu mensuel, charges mensuelles.
credit_files	identifiant, référence du client, auteur, type de crédit, montant, revenu du co-emprunteur, durée, objet, taux, apport, crédits en cours, mensualité, historique de crédit, garantie, statut, score de risque, niveau de risque, décision du modèle, facteurs de décision, avis de l'agent, commentaires.
audit_logs	identifiant, matricule de l'auteur, action, entité concernée, identifiant de l'entité, description, horodatage.

Les **facteurs de décision** constituent un document imbriqué dans le dossier, comportant pour chaque critère son nom, son impact chiffré et le sens de son influence. Ce choix d'imbrication plutôt que de collection séparée se justifie par le cycle de vie de ces données : elles n'ont aucune existence indépendante du dossier et sont toujours lues avec lui.

3.3 Conception des interfaces

L'interface s'organise autour d'un gabarit commun : une barre latérale de navigation, une barre supérieure portant l'identité de l'utilisateur connecté, et une zone de contenu. Les écrans d'authentification échappent à ce gabarit et adoptent une présentation en deux volets, formulaire et visuel institutionnel.

Table 3.2 – Principaux écrans

Écran	Contenu
Connexion / Inscription	Formulaire avec validation par champ, transition animée entre les deux vues.
Tableau de bord	Cartes d'indicateurs et graphiques de répartition.
Liste des clients / dossiers	Tableau filtrable, actions de consultation, modification et suppression.
Détail d'un dossier	Informations générales, modalités, garanties, avis de l'agent, et panneau d'analyse.
Panneau d'analyse	Score, niveau, décision, puis restitution des critères déterminants sous forme de barres proportionnelles.
Journal d'audit	Tableau chronologique, accessible aux seuls administrateurs.

Chapitre 4

Réalisation

4.1 Environnement de travail

Table 4.1 – Environnement logiciel

Catégorie	Outils
Développement	IntelliJ IDEA, Visual Studio Code, Jupyter Notebook
Gestion de version	Git, GitHub
Conteneurisation	Docker, Docker Compose
Orchestration	Kubernetes, Terraform
Supervision	Prometheus, Grafana
Qualité et sécurité	SonarCloud, Trivy, CodeQL, Gitleaks, OWASP ZAP
Tests	JUnit 5, Mockito, Jasmine, Karma, pytest

4.2 Technologies retenues

Table 4.2 – Choix technologiques et justifications

Couche	Technologie	Justification
Frontend	Angular 19	Framework structurant imposant une organisation claire ; typage statique ; composants autonomes et signaux pour la réactivité.
Backend	Spring Boot 4.1 / Java 21	Écosystème mature ; Spring Security fournit l'authentification et les autorisations sans ré-implémentation.
Base	MongoDB 7	Schéma souple, adapté à l'évolution des critères d'analyse.
Analyse	FastAPI / Python 3.11	Accès direct à l'écosystème scikit-learn ; validation automatique des entrées par le typage.
Modèle	Régression logistique	Modèle linéaire dont les coefficients sont directement interprétables — condition nécessaire à l'explicabilité recherchée.

4.3 Sécurité applicative

L'authentification est **sans état** : après vérification des identifiants, le serveur émet un jeton JWT signé, transmis par le client à chaque requête. Le serveur ne conserve aucune session, ce qui permet de multiplier les instances du backend sans partage d'état.

Les mots de passe sont hachés avec BCrypt, fonction à coût paramétrable conçue pour résister aux attaques par force brute.

Les autorisations sont vérifiées **côté serveur**, et non seulement masquées dans l'interface. Un appel à `/api/audit-logs` par un compte non administrateur reçoit une réponse 403, indépendamment de ce que l'interface affiche. Ce point a été vérifié explicitement : masquer un menu ne protège pas la ressource qu'il désigne.

La clé de signature n'est jamais versionnée. Elle est fournie au démarrage par une variable d'environnement, et l'application refuse de démarrer si elle est absente ou d'une longueur insuffisante — un secret manquant provoque ainsi une erreur immédiate et explicite, plutôt qu'une défaillance silencieuse à la première connexion.

```

1 @PostConstruct
2 void validateSecret() {
3     if (secret == null || secret.length() < MIN_SECRET_LENGTH) {
4         throw new IllegalStateException(
5             "jwt.secret is missing or too short. Set the JWT_SECRET "
6             + "environment variable to at least "
7             + MIN_SECRET_LENGTH + " characters.");
8     }
9 }

```

Listing 4.1 – Validation de la clé de signature au démarrage

4.4 Stratégie de tests

Table 4.3 – Couverture de tests par service

Service	Tests	Outils
Backend	36	JUnit 5, Mockito, JaCoCo
Frontend	49	Jasmine, Karma
Service d'analyse	7	pytest, pytest-cov
Total	92	

Les tests ne visent pas une couverture maximale mais les points où une régression aurait des conséquences réelles :

- la **conversion des données du dossier** vers les variables attendues par le modèle ;
- la **sécurité du jeton** : rejet d'un jeton falsifié, expiré ou signé avec une autre clé ;
- les **gardes métier** : unicité du numéro de carte d'identité, traitement des entités inexistantes ;
- le **contrat du service d'analyse** : structure de la réponse, cohérence entre score et niveau, ordonnancement des facteurs.

Le premier point mérite d'être détaillé, car il provient d'une anomalie réelle. Le jeu de données d'entraînement exprime le montant du prêt en milliers, tandis que l'application le stocke en dinars. Cette différence d'échelle, non détectée initialement, conduisait le modèle à considérer tous les montants comme démesurés : chaque dossier recevait un avis défavorable. Un test fige désormais cette conversion.

```
1 @Test
2 void convertsLoanAmountFromTndToThousandsForTheModel() {
3     MlPredictionRequest request = captureRequest(
4         baseClient().build(), baseFile().build());
5
6     // 128 000 TND est transmis au modele sous la forme 128
7     assertThat(request.getLoanAmount()).isEqualTo(128.0);
8 }
```

Listing 4.2 – Test de la conversion d'échelle du montant

4.5 Interfaces réalisées

Authentification

L'accès à la plateforme s'effectue par matricule et mot de passe. L'écran adopte une présentation en deux volets : le formulaire à gauche, un visuel institutionnel à droite. Chaque champ est validé à la saisie, avec un message explicite en cas d'erreur.

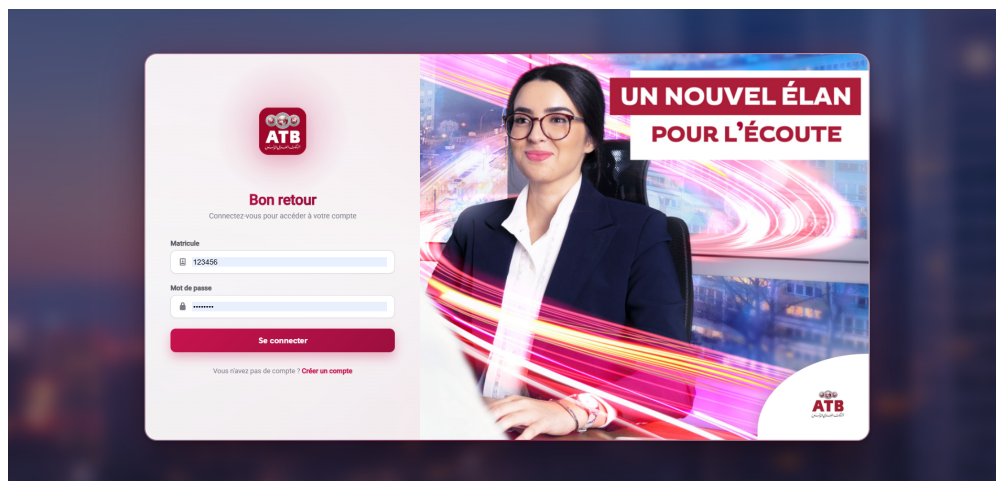


Figure 4.1 – Écran de connexion

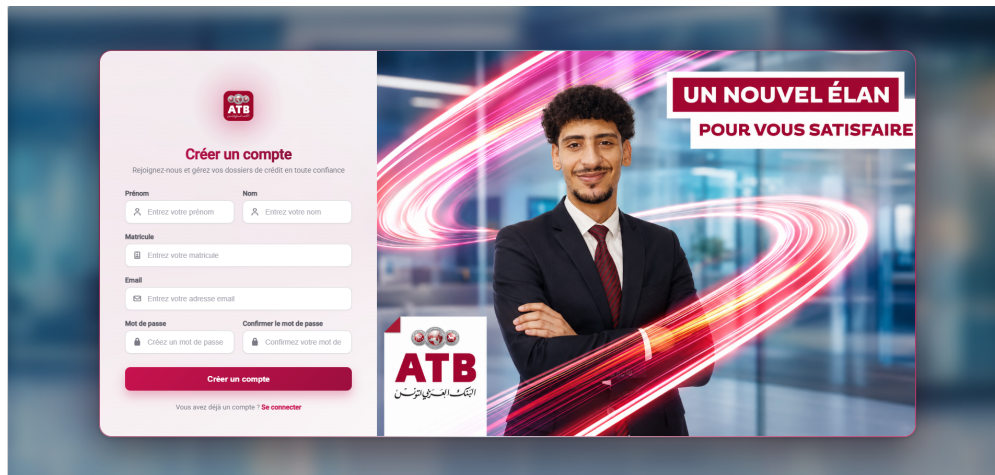


Figure 4.2 – Écran de création de compte, avec validation par champ

Analyse d'un dossier de crédit

L'écran de détail rassemble les informations du dossier et le panneau d'analyse. Ce dernier affiche le score, le niveau de risque et la décision indicative, puis la décomposition de cette décision : chaque critère est représenté par une barre dont la longueur traduit son poids relatif, et dont la couleur et la flèche indiquent s'il abaisse ou élève le risque.

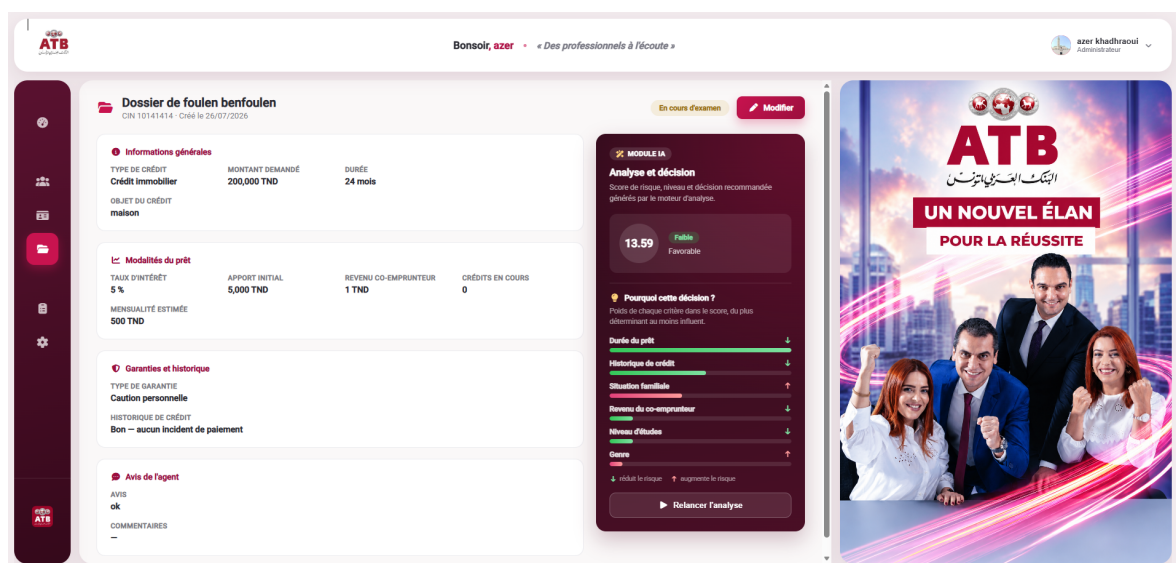


Figure 4.3 – Détail d'un dossier et panneau d'analyse avec explicabilité

C'est cette restitution qui distingue la plateforme d'un simple calculateur de score : l'agent dispose non seulement d'un résultat, mais des éléments qui le justifient.

Journal d’audit

Toute action sensible est enregistrée avec son auteur, sa nature, l’entité concernée et son horodatage. La consultation est réservée aux administrateurs : un agent qui appellerait directement l’API correspondante reçoit une réponse 403.

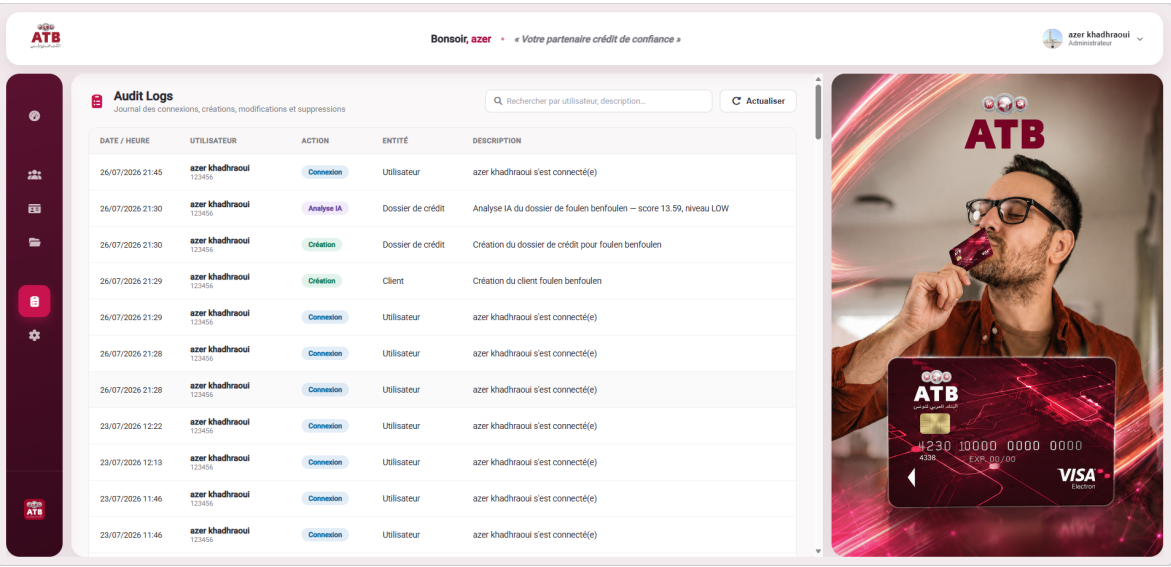


Figure 4.4 – Journal d’audit — traçabilité des connexions, créations et analyses

Chapitre 5

Module d'apprentissage automatique

5.1 Jeu de données

Le modèle est entraîné sur le jeu de données public *Loan Prediction*, qui décrit des demandes de crédit et leur issue. Chaque observation comporte onze variables explicatives et une variable cible binaire indiquant si le prêt a été accordé.

Table 5.1 – Variables du jeu de données

Variable	Type	Description
Gender	catégorielle	Genre du demandeur
Married	catégorielle	Situation matrimoniale
Dependents	numérique	Personnes à charge (0 à 3, « 3+ » regroupé)
Education	catégorielle	Diplômé ou non diplômé
Self_Employed	catégorielle	Travailleur indépendant
ApplicantIncome	numérique	Revenu du demandeur
CoapplicantIncome	numérique	Revenu du co-emprunteur
LoanAmount	numérique	Montant demandé (en milliers)
Loan_Amount_Term	numérique	Durée en mois
Credit_History	binaire	Historique conforme aux critères (0 ou 1)
Property_Area	catégorielle	Zone urbaine, semi-urbaine ou rurale

Ce jeu de données présente deux caractéristiques qui ont orienté le traitement. D'une part, plusieurs variables comportent des valeurs manquantes, ce qui impose une stratégie d'imputation avant tout entraînement. D'autre part, les échelles diffèrent fortement d'une variable à l'autre : les revenus s'expriment en unités, le montant du prêt en milliers. Cette dernière particularité est à l'origine de l'anomalie décrite au chapitre précédent.

5.2 Préparation des données

Le traitement appliqué avant l'entraînement comprend le remplacement des valeurs manquantes, l'encodage des variables catégorielles en valeurs numériques, et la séparation du jeu en ensembles d'entraînement et de test.

Les encodeurs ajustés lors de l'entraînement sont **sérialisés avec le modèle**. Ce point est essentiel à la cohérence du système : le service d'analyse doit appliquer exactement la même transformation que celle utilisée à l'entraînement, sous peine d'interpréter les catégories différemment. Les moyennes de chaque variable sont également conservées, car elles servent de référence au calcul des explications.

5.3 Choix du modèle

Une **régression logistique** a été retenue. Ce choix résulte d'un arbitrage explicite entre performance brute et interprétabilité.

Des modèles d'ensemble auraient pu atteindre des scores légèrement supérieurs. Mais leurs décisions ne sont pas décomposables : on peut mesurer l'importance globale d'une variable, non expliquer une prédiction particulière sans recourir à des méthodes d'approximation supplémentaires.

Or l'objectif du projet n'est pas de maximiser une métrique, mais de **fournir à l'agent une décision qu'il puisse comprendre et justifier auprès du client**. Un modèle linéaire expose directement ses coefficients : la contribution de chaque variable se calcule exactement, sans approximation. La performance légèrement moindre est le prix assumé de cette transparence.

Le modèle a été évalué sur un ensemble de test tenu à l'écart de l'entraînement. Sa capacité à discriminer les dossiers se vérifie également en production : les deux cas présentés à la section suivante, l'un scoré à 73,68 et l'autre à 9,60, correspondent bien à des profils de risque distincts, et la décomposition de leurs scores confirme que le modèle s'appuie sur les critères attendus — l'historique de crédit en premier lieu.

5.4 Explicabilité des décisions

C'est la contribution distinctive du projet. Pour chaque dossier analysé, le service ne retourne pas seulement un score, mais la **contribution de chaque variable** à ce score, exprimée en log-odds :

$$contribution_i = \beta_i \times (x_i - \bar{x}_i)$$

où β_i désigne le coefficient de la variable i , x_i sa valeur pour le dossier analysé et $\bar{x}_i$ sa moyenne sur l'ensemble d'entraînement.

La pondération par l'écart à la moyenne est ce qui rend l'explication **spécifique au dossier**. Sans elle, on décrirait le comportement général du modèle ; avec elle, on décrit ce qui distingue ce demandeur précis d'un profil moyen. Une contribution positive abaisse le risque, une contribution négative l'élève.

```

1 def _explain(self, df):
2     coefficients = self.model.coef_[0]
3     row = df.iloc[0]
4     factors = []
5     for column, coefficient in zip(self.feature_columns, coefficients)
6         :
7         impact = float(coefficient * (row[column] - self.feature_means
8             [column]))
9         factors.append({
10             "feature": column,
11             "impact": round(impact, 4),
12             "reduces_risk": impact > 0,
13         })
14     factors.sort(key=lambda f: abs(f["impact"]), reverse=True)
15     return factors

```

Listing 5.1 – Calcul des facteurs de décision

Les six facteurs les plus déterminants sont conservés et transmis à l'interface, qui les restitue sous forme de barres ordonnées dont la longueur est proportionnelle au facteur dominant (figure 4.3). Ce choix de représentation relative répond à une contrainte d'usage : une valeur de log-odds n'a aucune signification pour un agent, alors qu'un poids relatif — « l'historique de crédit pèse trois fois plus que tout le reste » — lui fournit un argument directement utilisable.

Table 5.2 – Explication produite pour un dossier à risque élevé (score 73,68)

Critère	Impact	Effet sur le risque
Historique de crédit	−2,40	élève
Durée du prêt	+0,77	abaisse
Niveau d'études	−0,35	élève
Situation familiale	+0,16	abaisse
Revenu du co-emprunteur	−0,08	élève
Genre	−0,06	élève

Table 5.3 – Explication produite pour un dossier à risque faible (score 9,60)

Critère	Impact	Effet sur le risque
Durée du prêt	+0,66	abaisse
Historique de crédit	+0,41	abaisse
Situation familiale	+0,16	abaisse
Niveau d'études	+0,10	abaisse
Genre	−0,06	élève
Personnes à charge	+0,05	abaisse

La comparaison des deux tableaux illustre l'intérêt de la démarche : dans le premier cas, un seul critère — l'historique de crédit — domine tous les autres et suffit à expliquer le rejet ; dans le second, plusieurs critères modérément favorables se cumulent. Cette différence de structure est invisible dans un score isolé.

5.5 Intégration au backend

Le backend convertit les données du dossier et du client en variables attendues par le modèle, appelle le service d'analyse, puis enregistre le résultat.

Trois difficultés d'intégration ont dû être résolues :

Conflit de bibliothèques de sérialisation

La version du framework backend employait une nouvelle génération de la bibliothèque de sérialisation JSON, tandis qu'une dépendance transitive en imposait l'ancienne. Les deux coexistaient dans le projet, provoquant des erreurs à l'exécution. La résolution a consisté à sélectionner explicitement l'implémentation retenue et à lui appliquer la

convention de nommage du service Python, plutôt que d'annoter chaque champ individuellement.

Négociation du protocole HTTP

Le client HTTP du langage tente par défaut une montée de version du protocole. Le serveur Python rejetait cette négociation, ce qui entraînait la perte du corps de la requête : le service recevait un appel vide. Le diagnostic a nécessité l'examen des journaux du serveur, l'erreur n'étant pas visible côté client. La correction consiste à imposer explicitement la version du protocole.

Incohérence d'échelle

Décrite au chapitre précédent, cette anomalie faussait toutes les prédictions. Elle illustre un point méthodologique : un système peut fonctionner sans erreur technique tout en produisant des résultats systématiquement faux. Seule la confrontation des prédictions à des cas dont l'issue était attendue a permis de la détecter.

Démarche DevSecOps et déploiement

6.1 Conteneurisation

Chaque service dispose de son image Docker, construite en plusieurs étapes : une première compile l'application, une seconde ne conserve que le résultat. Les outils de compilation n'apparaissent donc pas dans les images livrées, ce qui réduit à la fois leur taille et leur surface d'attaque.

Deux mesures complètent ce durcissement : les processus s'exécutent sous un utilisateur non privilégié, et les images de base retenues sont des variantes minimales.

L'orchestration locale par Docker Compose décrit les quatre conteneurs, leur réseau commun, leurs volumes persistants et leur ordre de démarrage. Ce dernier point s'appuie sur des sondes de santé : le backend n'attend pas que la base et le service d'analyse soient *lancés*, mais qu'ils *répondent*.

6.2 Intégration continue

Table 6.1 – Étapes de la chaîne d'intégration continue

Étape	Rôle
Backend	Compilation et exécution des 36 tests, avec une instance MongoDB dédiée fournie pour la durée de la chaîne.
Frontend	Exécution des 49 tests dans un navigateur sans interface, puis compilation en configuration de production.
Service d'analyse	Exécution des 7 tests contre le modèle entraîné réel, vérifiant le contrat de l'API et la cohérence des résultats.
Qualité	Analyse SonarCloud consolidant les rapports de couverture des trois services et vérification du seuil de qualité.
Images	Construction des trois images Docker, validant les fichiers de construction.

6.3 Sécurité intégrée à la chaîne

La sécurité n'est pas traitée comme un audit de fin de cycle : elle s'exécute au même déclencheur que les tests, à chaque modification. C'est précisément ce qui distingue une démarche **DevSecOps** d'une intégration continue à laquelle on adjoindrait une revue de sécurité ponctuelle.

Table 6.2 – Outils d'analyse et périmètres couverts

Outil	Type	Périmètre
Gitleaks	Secrets	Intégralité de l'historique du dépôt, et non le seul état courant : un secret supprimé demeure présent dans les révisions antérieures.
CodeQL	SAST	Analyse statique du code source, en Java, TypeScript et Python.
Trivy	SCA	Vulnérabilités connues des dépendances déclarées.
Trivy	Image	Vulnérabilités des paquets système présents dans les images.
OWASP ZAP	DAST	Application réellement démarrée, sollicitée depuis l'extérieur.
SonarCloud	Qualité	Couverture de tests, duplication, maintenabilité.

Ces analyses sont complémentaires et non redondantes. L'analyse statique examine le code sans l'exécuter et ne peut rien dire d'une faille située dans une bibliothèque tierce. L'analyse de composition inspecte les dépendances mais ignore les défauts de la logique applicative. L'analyse dynamique éprouve l'application en fonctionnement et révèle ce que les deux précédentes ne peuvent voir : en-têtes de sécurité absents, points d'entrée exposés, comportements à l'exécution. Aucune ne couvre le périmètre des deux autres.

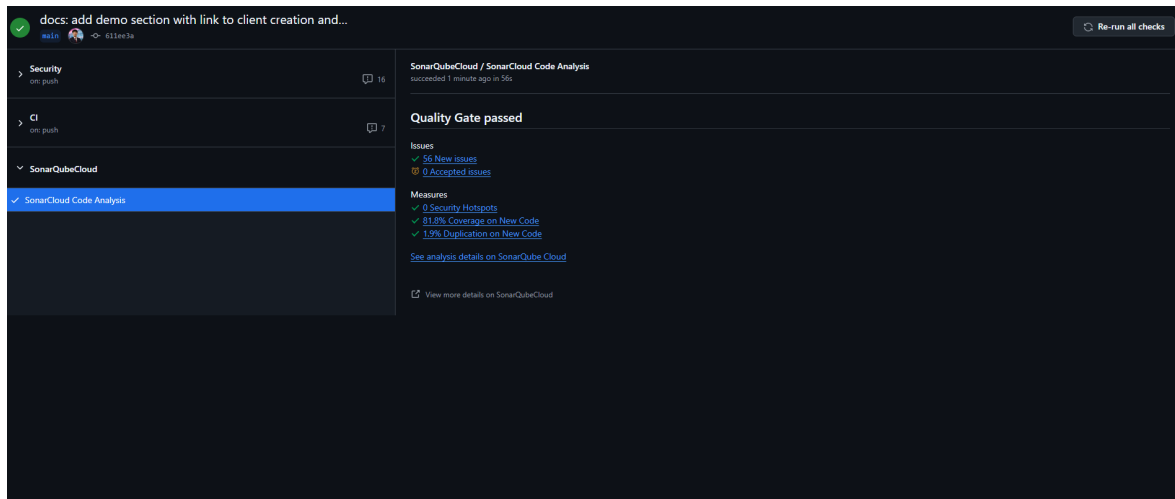


Figure 6.1 – Exécution de la chaîne d'intégration continue et des analyses de sécurité

Les résultats sont publiés dans l'espace de sécurité du dépôt au format standardisé SARIF. Une analyse hebdomadaire est également programmée : de nouvelles vulnérabilités peuvent être publiées contre un code demeuré inchangé.

Un choix mérite d'être explicité : les analyses **signalent sans bloquer**. Une vulnérabilité transitive dépourvue de correctif ne doit pas empêcher une livraison ; elle doit être connue et suivie. Bloquer systématiquement conduirait, en pratique, à désactiver l'analyse.

6.4 Déploiement sur Kubernetes

Le déploiement sur cluster repose sur une description **déclarative** : les manifestes n'énoncent pas une séquence d'actions, mais l'état attendu du système. L'orchestrateur compare en permanence l'état réel à l'état déclaré et corrige les écarts. C'est la différence de nature avec Docker Compose, qui lance des conteneurs sans maintenir d'objectif.

Table 6.3 – Objets Kubernetes déployés et justifications

Service	Objet	Justification
MongoDB	StatefulSet	Une base de données possède un état : ses instances ne sont pas interchangeables. Le StatefulSet attribue une identité stable et lie chaque instance à son propre volume.
Service d'analyse	Deployment	Service sans état, exposé uniquement à l'intérieur du cluster : il n'est appelé que par le backend. Restreindre son exposition réduit la surface d'attaque.
Backend	Deployment	Exposé à l'extérieur, l'application cliente s'adressant à l'API depuis le navigateur de l'utilisateur.
Frontend	Deployment	Exposé à l'extérieur pour l'accès des utilisateurs.

La configuration non sensible provient d'un *ConfigMap*, la clé de signature d'un *Secret* créé hors du dépôt — prolongement de la règle appliquée en développement.

Trois sondes distinctes gouvernent le cycle de vie des conteneurs :

- la sonde de **démarrage** protège la phase d'initialisation ; sans elle, un service lent à démarrer serait considéré comme défaillant et redémarré indéfiniment ;
- la sonde d'**aptitude** retire du service un conteneur temporairement incapable de répondre, sans le détruire ;
- la sonde de **vitalité** provoque son remplacement lorsqu'il est durablement bloqué.

Enfin, l'orchestrateur ne disposant pas de mécanisme natif d'ordonnancement entre services, des conteneurs d'initialisation retiennent le démarrage du backend jusqu'à ce que ses dépendances acceptent les connexions.

6.5 Infrastructure décrite en code

Les manifestes décrivent déjà l'état attendu du système, mais leur application repose sur une commande qui ne rend compte que de ce qu'elle vient de faire. Terraform apporte une garantie supplémentaire : avant toute modification, il compare la configuration décrite, l'état qu'il a enregistré et la réalité du cluster, puis affiche précisément ce qui va changer.

```

1 # kubernetes_deployment.ml_service will be updated in-place
2   ~ replicas = "1" -> "2"
3

```

```
4 Plan: 0 to add, 1 to change, 0 to destroy.
```

Listing 6.1 – Aperçu produit avant application d’une modification

Cette prévisualisation est ce qui distingue l’outil : elle transforme une modification d’infrastructure en opération vérifiable avant exécution.

Terraform conserve la trace de ce qu’il a créé dans un fichier d’état, qui constitue sa seule mémoire — il ne découvre pas le cluster, il consulte son propre registre. C’est pourquoi il gère un espace de noms distinct de celui déployé par les manifestes : reprendre la gestion de ressources créées par un autre moyen supposerait de les lui déclarer une à une.

6.6 Supervision applicative

Le déploiement ne suffit pas à savoir si un service se comporte correctement une fois en fonctionnement. Une chaîne de supervision complète le dispositif.

Le backend expose ses métriques d’exécution sur un point d’entrée dédié : consommation mémoire, charge processeur, nombre de fils d’exécution, mais aussi volume et durée des requêtes par route. Prometheus les collecte toutes les quinze secondes et en conserve l’historique. Grafana interroge ensuite Prometheus et restitue ces données sous forme de tableaux de bord.



Figure 6.2 – Tableau de bord de supervision du backend

Un point mérite d’être souligné : la source de données et le tableau de bord ne sont pas configurés manuellement dans l’interface de Grafana, mais décrits dans des fichiers

montés au démarrage. Un réglage effectué à la main disparaîtrait au premier redémarrage du conteneur, puisque celui-ci ne conserve pas d'état ; sous forme de fichier, il est reproduit à chaque déploiement et versionné avec le reste du projet.

L'exposition suit la même règle que le reste de l'architecture : Prometheus n'est accessible qu'à l'intérieur du cluster, seul Grafana l'interroge, et seule l'interface de Grafana est publiée.

6.7 Publication des images

La publication des images est déclenchée par la pose d'une étiquette de version sur le dépôt. Les images sont alors construites et publiées sur un registre, étiquetées à la fois par le numéro de version et par la mention de dernière version disponible.

Cette distinction est importante. Une étiquette mobile ne change pas de nom lorsque son contenu change : rien ne signale à l'orchestrateur qu'il doit rafraîchir son cache local, et une version obsolète peut continuer d'être servie malgré une reconstruction. Le déploiement reproductible impose donc des étiquettes immuables.

Conclusion générale

Bilan du travail réalisé

Les objectifs fixés en introduction ont été atteints. La plateforme livrée centralise la gestion des clients et de leurs dossiers de crédit, calcule un score de risque à partir d'un modèle entraîné, et — c'est sa contribution distinctive — restitue à l'agent les critères qui ont déterminé ce score, classés par influence. La traçabilité est assurée par un journal d'audit couvrant chaque opération sensible, dont la consultation est réservée aux administrateurs.

Le déploiement a été mené jusqu'au bout de la chaîne : conteneurisation des trois services, orchestration sur cluster Kubernetes, description de l'infrastructure en code, supervision des métriques d'exécution. Une chaîne d'intégration continue exécute à chaque modification les 92 tests automatisés et six analyses de sécurité couvrant le code, les dépendances, les images et l'application en fonctionnement.

Le résultat n'est donc pas une maquette illustrant un concept, mais une application complète, testée et déployable.

Apports du stage

Sur le plan technique, ce projet a été l'occasion de mettre en œuvre un ensemble d'outils qui m'étaient inconnus au début du stage : la conteneurisation et son orchestration, la description d'infrastructure en code, la supervision applicative, et l'intégration d'analyses de sécurité automatisées dans une chaîne de livraison. Chacun a été abordé non comme une fin en soi, mais parce qu'il répondait à un besoin concret du projet.

L'apport méthodologique me paraît toutefois plus déterminant, et tient en trois enseignements.

Un arbitrage explicite vaut mieux qu'une optimisation aveugle. Le choix de la régression logistique s'est fait contre des modèles susceptibles d'obtenir de meilleurs scores. Ce choix ne se justifie que par l'usage visé : un agent bancaire ne peut se servir d'un verdict qu'il ne peut expliquer à son client. La métrique la plus élevée n'est pas toujours le bon critère de décision.

Un système peut fonctionner sans erreur et produire des résultats faux.

L'anomalie d'échelle sur le montant du prêt en offre l'illustration la plus nette : aucune exception n'était levée, tous les composants répondaient correctement, et pourtant chaque dossier recevait un avis défavorable. Seule la confrontation des prédictions à des cas dont l'issue était connue a permis de la détecter. J'en retiens qu'un test vérifiant l'absence d'erreur ne vérifie pas la justesse du résultat.

Le diagnostic demande de chercher au bon endroit. Le problème de négociation du protocole HTTP restait invisible côté client : la requête paraissait partir normalement. Seule la lecture des journaux du serveur a révélé la cause. Une erreur ne se manifeste pas nécessairement là où elle se produit.

Limites et perspectives

Plusieurs limites doivent être reconnues :

- le modèle est entraîné sur un jeu de données public, dont la population ne correspond pas à celle de la clientèle de l'ATB ; un réentraînement sur données réelles constituerait le prolongement naturel du travail ;
- la couverture de tests, satisfaisante sur la logique métier, demeure partielle sur les écrans de l'interface ;
- le déploiement a été validé sur un cluster local mono-nœud, et non en environnement de production ;
- la supervision porte sur les métriques techniques du backend ; elle ne couvre ni les indicateurs métier, ni le service d'analyse.

Plusieurs prolongements se dessinent à partir de ces limites. Le premier, et le plus porteur de valeur, serait le réentraînement du modèle sur l'historique réel de la banque : les coefficients refléteraient alors les caractéristiques de sa clientèle, et l'explication produite gagnerait en pertinence.

Sur le plan de l'exploitation, la mise en place d'alertes sur les métriques déjà collectées permettrait de signaler une dégradation avant qu'un utilisateur ne la constate. Une limitation du nombre de tentatives de connexion renforcerait par ailleurs la protection contre les attaques par force brute.

Enfin, la qualité d'un modèle se dégrade à mesure que la population évolue. Un réentraînement périodique, accompagné d'un suivi des écarts entre prédictions et décisions effectives, serait nécessaire pour maintenir sa pertinence dans le temps.

Bibliographie

- [1] Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/>
- [2] Spring Security Reference. <https://docs.spring.io/spring-security/reference/>
- [3] Angular Documentation. <https://angular.dev/>
- [4] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [5] Pedregosa, F. et al. *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research, vol. 12, p. 2825–2830, 2011.
- [6] MongoDB Manual. <https://www.mongodb.com/docs/manual/>
- [7] Docker Documentation. <https://docs.docker.com/>
- [8] Kubernetes Documentation. <https://kubernetes.io/docs/>
- [9] Terraform Documentation. <https://developer.hashicorp.com/terraform/docs>
- [10] Prometheus Documentation. <https://prometheus.io/docs/>
- [11] OWASP Top Ten. <https://owasp.org/www-project-top-ten/>
- [12] Jones, M., Bradley, J., Sakimura, N. *JSON Web Token (JWT)*. RFC 7519, IETF, 2015. <https://datatracker.ietf.org/doc/html/rfc7519>
- [13] Loan Prediction Problem Dataset. <https://www.kaggle.com/datasets/altruistdelhite04/loan-prediction-problem-dataset>